

Análisis de datos

Introducción.....	2
Ejemplo con datos de análisis clínico.....	2
Exploración inicial.....	2
Mostramos el número de filas y columnas del dataset.....	3
Guarda los nombres de las columnas en una lista.....	3
Comprueba la viabilidad de los datos.....	3
Filtrado de datos. Dataframes.....	5
Crear subconjuntos.....	5
Añadir columnas personalizadas.....	6
Resumen estadístico.....	6
Consultas.....	8
Gráficos.....	12
De tartas.....	12
Histograma.....	14
Nube de puntos.....	15
Regresión lineal.....	17
Calculamos el error del modelo.....	19
Generar divisiones de los datos originales con train_test_split.....	20



El csv para el ejemplo se ha sacado de la página www.kaggle.com

No os quedéis con el tema médico de la diabetes: lo importante es que aprendáis a razonar con datos, detectar patrones, interpretar resultados y justificar conclusiones lógicas. No memorices explicaciones médicas, quédate con el proceso analítico

Introducción

Para crear un modelo de datos predictivo, es esencial preparar adecuadamente los datos. Esto implica limpiar y transformar los datos crudos para asegurarse de que sean adecuados para el análisis, tratando los valores faltantes, atípicos y errores que podrían sesgar las predicciones. También incluye combinar conjuntos de datos, crear nuevas variables y seleccionar las más relevantes para el modelo.

Una vez los datos están listos, se eligen los algoritmos adecuados para entrenar el modelo y hacer predicciones.

Finalmente, evaluar el rendimiento del modelo es fundamental para garantizar su fiabilidad. Utilizando métricas, se mide qué tan bien el modelo predice los resultados.

Vamos a realizar un análisis de datos a un dataset sobre diabetes en función de varias variables, como embarazos, glucosa, presión arterial, grosor de piel, insulina, IMC, índice de riesgo genético y edad. A lo largo del análisis, aportaremos observaciones sobre lo que vayamos obteniendo.

Ejemplo con datos de análisis clínico

Exploración inicial

Vamos a trabajar con el fichero **diabetes.csv** cuyas columnas son:

Embarazos	Glucosa	PresionArterial	GrosorPiel	Insulina	IMC	IndiceRiesgoGenetico	Edad	Resultado
-----------	---------	-----------------	------------	----------	-----	----------------------	------	-----------

En la vida real, los analistas de datos no trabajan aislados: colaboran con personas expertas en cada campo —como médicos, epidemiólogos o especialistas en cualquier otra área— para entender bien los datos y su contexto. Por eso, en el examen no voy a pedirlos que seáis expertos en el tema del dataset que utilice para hacer preguntas, sino que sepáis pensar como analistas de datos: observar, interpretar y sacar conclusiones claras y bien razonadas. Lo importante es que sepáis aplicar lo aprendido, usar el código con sentido y tener criterio para explicar lo que veis.

Mostramos el número de filas y columnas del dataset.

Saber el tamaño del dataset ayuda a evaluar su complejidad, requerimientos de procesamiento, y verificar que se cargó correctamente.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Leemos el fichero con los datos
datos = pd.read_csv("diabetes.csv")

#Mostramos número de filas y columnas con shape
print("Número de filas:", datos.shape[0])

        Número de filas: 768

print("Número de columnas:", datos.shape[1])

        Número de columnas: 9
```

Guarda los nombres de las columnas en una lista.

Esta acción facilita la automatización de algunos procesos (por ejemplo, iterar sobre columnas para generar gráficos o cálculos).

```
# Lista con los nombres de las columnas
columnas_diabetes = datos.columns.tolist()
print("Columnas:", columnas_diabetes)

Columnas: ['Embarazos', 'Glucosa', 'PresionArterial', 'GrosorPiel', 'Insulina', 'IMC', 'IndiceRiesgoGenetico', 'Edad', 'Resultado']
```

Comprueba la viabilidad de los datos.

Cuando analizamos datos relacionados con la salud, es esencial tener en cuenta el contexto médico que nos den de las variables. Algunos valores, como 0 en IMC, son imposibles o inválidos en la realidad, y su presencia puede indicar errores en la medición o la entrada de datos. Además, ciertos valores pueden estar fuera de rango y necesitan ser revisados o corregidos, ya que podrían distorsionar los análisis y conclusiones. Siempre es crucial contar con el conocimiento del dominio (en este caso, salud) para identificar y manejar estos valores de manera adecuada.

Vamos a empezar detectando si hay valores nulos/faltantes en el fichero.

```
# Comprobamos existencia de valores nulos
print("Valores nulos por columna:\n", datos.isnull().sum())
```

```
Valores nulos por columna:
Embarazos          0
Glucosa            0
PresionArterial    0
GrosorPiel         0
Insulina           0
IMC                0
IndiceRiesgoGenetico 0
Edad              0
Resultado          0
dtype: int64
```

No hay ningún valor faltante.

Ahora vamos a suponer que un profesional de la salud nos ha dicho que los valores de glucosa, el índice de masa corporal y la presión arterial no pueden ser cero y que tenemos que descartar esos casos. Por lo tanto, vamos a quitar las filas donde ocurra esto.

```
# Nos quedamos con las filas sin 0 en columnas críticas
datos_preparados = datos[
    (datos["Glucosa"] != 0) &
    (datos["PresionArterial"] != 0) &
    (datos["IMC"] != 0)
].copy()

print(f"Filas después de limpieza: {len(datos_preparados)}")

Filas después de limpieza: 724
```

También vamos a rellenar los valores de insulina y de grosor de piel que están a cero para el mismo paciente (en la misma fila). Es probable que esos ceros representen datos faltantes no registrados, donde el personal médico ingresó "0" por error o para indicar que no se midió el valor. El valor que vamos a insertar es la mediana de los valores de esa misma columna, así mantenemos la integridad de los datos sin asumir valores arbitrarios, preservando la distribución real de las mediciones válidas.

Solo vamos a modificar las filas donde ambos valores coincidan en cero para no cambiar demasiados valores y evitar subestimar la variabilidad natural de los datos.

```
# Sacamos las filas a rellenar
ambos = (datos_preparados["Insulina"] == 0) &
        (datos_preparados["GrosorPiel"] == 0)

print("Filas con ceros en insulina y grosor:",
      datos_preparados[ambos].shape[0])

      Filas con ceros en insulina y grosor: 192

# Rellenamos ceros con medianas
medianas = datos_preparados[["Insulina", "GrosorPiel"]].median()
datos_preparados.loc[ambos, "Insulina"] = medianas["Insulina"]
datos_preparados.loc[ambos, "GrosorPiel"] = medianas["GrosorPiel"]
```

Filtrado de datos. Dataframes

Crear subconjuntos.

Vamos a crear un DataFrame, llamado **df_diabeticos**, que contenga solo las filas donde Resultado=1 (pacientes diagnosticados con diabetes), excluyendo la columna Resultado.

La segmentación de datos quita datos irrelevantes para hacer análisis específicos. En este caso concreto, nos permite estudiar características específicas de pacientes diabéticos sin "ruido" de otros grupos. Además, reduce el tiempo de procesamiento en datasets muy grandes con muchos datos.

```
# Nuevo dataframe solo con los datos de diabeticos
df_diabeticos = datos_preparados[datos_preparados["Resultado"] ==
1]
df_diabeticos = df_diabeticos.drop("Resultado", axis="columns")

print(df_diabeticos)
```

	Embarazos	Glucosa	PresionArterial	...	IMC	IndiceRiesgoGenetico	Edad
0	1	199	76	...	42.9	1.39	22
1	0	198	66	...	41.3	0.50	28
2	2	197	70	...	30.5	2.33	53
3	8	197	74	...	25.9	1.19	39
4	2	197	70	...	34.7	0.58	62
..	...	...	...	...	...	...	...
261	1	88	30	...	55.0	0.38	26
262	5	85	74	...	29.0	0.38	32
263	12	84	72	...	29.7	0.46	46
264	3	80	82	...	34.2	0.18	27
265	3	78	50	...	31.0	0.25	26

[249 rows x 8 columns]

Añadir columnas personalizadas.

Añade a **df_diabeticos** una nueva columna llamada **Relacion_Insulina_Glucosa** que contendrá los valores obtenidos con el siguiente cálculo:

$$\text{Relacion_insulina_glucosa} = \frac{\text{Insulina}}{\text{Glucosa}}$$

Una relación alta (Insulina alta / Glucosa alta) sugiere que el cuerpo produce insulina, pero no la utiliza eficientemente. Una relación baja (Insulina baja / Glucosa alta) puede indicar deficiencia de insulina, típica de diabetes tipo 1. Esto por ejemplo ayudaría a identificar pacientes con resistencia a la insulina.

```
# Usamos np.where para evitar apply (más rápido)
# Si glucosa vale cero, pondrá 0 para evitar dividir por cero
# Si no, hará la división

df_diabeticos["Relacion_insulina_glucosa"] = np.where(
    df_diabeticos["Glucosa"] == 0,
    0,
    df_diabeticos["Insulina"] / df_diabeticos["Glucosa"]
)

print(df_diabeticos)
```

	Embarazos	Glucosa	...	Edad	Relacion_insulina_glucosa
0	1	199	...	22	0.000000
1	0	198	...	28	1.383838
2	2	197	...	53	2.756345
3	8	197	...	39	0.243655
4	0	197	...	28	0.000000

Aunque en este caso no van a aparecer valores de glucosa igual a cero, porque quitamos esas filas, está bien tener en cuenta esa posibilidad para evitar fallos en la división.

Resumen estadístico

Con el resumen estadístico conseguimos una visión general de las variables. Por ejemplo, podemos mirar el valor medio de la glucosa de pacientes diabéticos. Vamos a ver el resumen estadístico del DataFrame original y el de diabéticos.

```
# Para ver las estadísticas de todas las columnas
```

```
pd.options.display.max_columns = None
print(df_diabeticos.describe())
```

	Embarazos	Glucosa	PresionArterial	GrosorPiel	Insulina \
count	249.000000	249.000000	249.000000	249.000000	249.000000
mean	4.907631	142.610442	75.248996	30.381526	121.871486
std	3.708575	30.058895	12.291579	9.660101	131.695885
min	0.000000	78.000000	30.000000	7.000000	0.000000
25%	2.000000	119.000000	68.000000	24.000000	48.000000
50%	4.000000	142.000000	74.000000	28.000000	64.000000
75%	8.000000	168.000000	84.000000	36.000000	175.000000
max	17.000000	199.000000	114.000000	99.000000	846.000000

	IMC	IndiceRiesgoGenetico	Edad	Relacion_insulina_glucosa
count	249.000000	249.000000	249.000000	249.000000
mean	35.313655	0.522811	37.341365	0.838187
std	6.589824	0.374641	11.122120	0.836536
min	22.900000	0.090000	21.000000	0.000000
25%	30.900000	0.260000	28.000000	0.289157
50%	34.300000	0.410000	36.000000	0.505263
75%	38.500000	0.690000	45.000000	1.210145
max	67.100000	2.330000	70.000000	4.838710

```
print(datos_preparados.describe())
```

	Embarazos	Glucosa	PresionArterial	GrosorPiel	Insulina \
count	724.000000	724.000000	724.000000	724.000000	724.000000
mean	3.866022	121.882597	72.400552	27.808011	97.223757
std	3.362803	30.750030	12.379870	9.304910	109.491429
min	0.000000	44.000000	24.000000	7.000000	0.000000
25%	1.000000	99.750000	64.000000	24.000000	48.000000
50%	3.000000	117.000000	72.000000	24.000000	48.000000
75%	6.000000	142.000000	80.000000	33.000000	130.500000
max	17.000000	199.000000	122.000000	99.000000	846.000000

	IMC	IndiceRiesgoGenetico	Edad	Resultado
count	724.000000	724.000000	724.000000	724.000000
mean	32.467127	0.471464	33.350829	0.343923
std	6.888941	0.322964	11.765393	0.475344
min	18.200000	0.080000	21.000000	0.000000
25%	27.500000	0.250000	24.000000	0.000000
50%	32.400000	0.370000	29.000000	0.000000
75%	36.600000	0.622500	41.000000	1.000000
max	67.100000	2.330000	81.000000	1.000000

Se ve también que la media de glucosa en el dataframe de diabéticos es mayor que la media del dataframe general.

Consultas

¿Cuál es el porcentaje de diabéticos y no diabéticos en el dataset?

```
#Calcular porcentaje de diabéticos y no diabéticos
conteo = datos_preparados["Resultado"].value_counts(normalize=True)*100

print(f"- No diabéticos (Resultado=0): {conteo[0]:.2f}%")
print(f"- Diabéticos (Resultado=1): {conteo[1]:.2f}%")

- No diabéticos (Resultado=0): 65.61%
- Diabéticos (Resultado=1): 34.39%
```

¿Cómo se distribuyen los pacientes diabéticos por grupos de edad (Edad en rangos: 20-29, 30-39, etc.)?

```
# Grupos de edad
bins = [20, 30, 40, 50, 60, 70, 80, 90]
labels = ["20-29", "30-39", "40-49", "50-59", "60-69", "70-79", "80+"]

df_diabeticos["GrupoEdad"] = pd.cut(df_diabeticos["Edad"], bins=bins,
                                     labels=labels, right=False)

# Contamos casos por grupo de edad
distribucion = df_diabeticos["GrupoEdad"].value_counts().sort_index()

# Calculamos porcentajes
porcentajes = (distribucion / len(df_diabeticos) * 100).round(2)

print(pd.DataFrame({"Casos": distribucion, "%": porcentajes}))
```

	Casos	%
GrupoEdad		
20-29	77	30.92
30-39	69	27.71
40-49	61	24.50
50-59	33	13.25
60-69	8	3.21
70-79	1	0.40
80+	0	0.00

¿Cuál es la media y desviación estándar de Glucosa en diabéticos vs no diabéticos? ¿En qué grupo es mayor?

```
# Media de Glucosa
print("\nMedia de Glucosa por Resultado:")
print(datos_preparados.groupby("Resultado")["Glucosa"].agg(["mean", "std"]))
```

Media de Glucosa por Resultado:		
	mean	std
Resultado		
0	111.016842	25.010709
1	142.610442	30.058895

Los diabéticos presentan una glucosa promedio más alta que los no diabéticos, lo que es esperado, ya que la glucosa elevada es uno de los principales indicativos de la diabetes. La mayor desviación estándar en los diabéticos también indica una mayor variabilidad en los niveles de glucosa dentro de este grupo.

¿Cuál es la media de IMC en pacientes diabéticos con más de 3 embarazos vs los que tienen 0 embarazos?

```
# Media IMC para diferentes cantidades de embarazos en diabéticos
print("\nIMC medio para personas diabéticas sin embarazos:")
print(df_diabeticos[df_diabeticos["Embarazos"] == 0]["IMC"].mean().round(2))
```

IMC medio de personas diabéticas sin embarazos:	
	39.35

```
print("\nIMC medio para personas diabéticas con más de 3 embarazos:")
print(df_diabeticos[df_diabeticos["Embarazos"] > 3]["IMC"].mean().round(2))
```

IMC medio de personas diabéticas con más de 3 embarazos:	
	34.5

Es probable que el cuidado durante el embarazo y el monitoreo médico de las mujeres diabéticas con más de tres embarazos esté contribuyendo a un IMC más bajo en comparación con las mujeres diabéticas sin embarazos. O que en los no embarazados se encuentren perfiles de varones. Normalmente las mujeres con más de 3 embarazos suelen tener mayor ganancia de peso acumulada, lo que eleva el IMC, pero en esta recopilación de datos no se da ese caso.

Cuál es la media de Insulina en pacientes con $\text{IndiceRiesgoGenetico} > 0.5$ vs $\text{IndiceRiesgoGenetico} < 0.5$

```
# Media de insulina por grupo de riesgo genético
media_alto = datos_preparados[datos_preparados["IndiceRiesgoGenetico"] >
                                0.5]["Insulina"].mean()
media_bajo = datos_preparados[datos_preparados["IndiceRiesgoGenetico"] <=
                                0.5]["Insulina"].mean()

print(f"Media de Insulina en pacientes con IndiceRiesgoGenetico > 0.5:
      {media_alto:.2f}")
      Media de Insulina en pacientes con IndiceRiesgoGenetico > 0.5: 107.14

print(f"Media de Insulina en pacientes con IndiceRiesgoGenetico ≤ 0.5:
      {media_bajo:.2f}")
      Media de Insulina en pacientes con IndiceRiesgoGenetico ≤ 0.5: 91.67
```

La insulina en sangre en ayunas generalmente se considera normal en el rango de 18 a 102 pmol/L.

¿Cómo varía la presión arterial en pacientes diabéticos según el número de embarazos?

```
# Estadísticas de presión arterial según el número de embarazos
presion_embarazos =
df_diabeticos.groupby("Embarazos")["PresionArterial"].agg(["mean", "std", "count"])

print("Presión arterial en diabéticos según el número de embarazos:")
print(presion_embarazos)
```

Presión arterial en diabéticos según el número de embarazos:								
	count	mean	std	min	25%	50%	75%	max
Embarazos								
0	32.0	75.062500	14.747198	40.0	66.0	74.0	85.75	110.0
1	28.0	73.857143	15.952973	30.0	64.0	74.0	86.50	102.0
2	18.0	72.888889	10.110501	52.0	68.0	70.0	80.00	90.0
3	26.0	70.769231	11.158164	50.0	64.0	72.0	78.00	92.0
4	21.0	73.380952	8.428975	60.0	70.0	72.0	78.00	92.0
5	20.0	78.800000	12.488310	62.0	73.5	76.0	82.50	108.0
6	15.0	73.066667	10.138094	50.0	70.0	74.0	79.00	92.0
7	24.0	74.083333	11.412871	50.0	66.0	75.0	84.00	90.0
8	20.0	77.800000	11.459769	62.0	71.5	76.0	80.00	106.0
9	18.0	82.055556	10.859922	56.0	78.0	82.0	86.00	110.0
10	9.0	73.888889	8.950481	62.0	68.0	70.0	84.00	86.0
11	6.0	82.000000	7.155418	74.0	77.0	82.0	84.00	94.0
12	4.0	71.500000	8.225975	62.0	68.0	71.0	74.50	82.0
13	4.0	91.500000	17.233688	72.0	85.5	90.0	96.00	114.0
14	2.0	70.000000	11.313708	62.0	66.0	70.0	74.00	78.0
15	1.0	70.000000	NaN	70.0	70.0	70.0	70.00	70.0
17	1.0	72.000000	NaN	72.0	72.0	72.0	72.00	72.0

No hay un aumento consistente de la presión arterial con mayor número de embarazos. Hay que tener en cuenta que los tamaños de muestra son muy pequeños para grupos con 10 o más embarazos, lo que hace menos confiables esas medias. En grupos con 9 o más embarazos, la mediana (50%) es cercana a la media, lo que sugiere una distribución simétrica.

La variabilidad/desviación típica (std) es alta en casi todos los grupos, indicando que hay mucha dispersión en los datos. Los grupos con mayor Rango Intercuartílico (IQR = 75% – 25%) muestran una mayor dispersión, lo cual concuerda con las elevadas desviaciones estándar. Por ejemplo en 0 embarazos, IQR = 19.75 y en 1 embarazo, IQR = 22.5.

¿Cuál es la media y desviación estándar del GrosorPiel en pacientes diabéticos vs no diabéticos?

```
#Media y desviación estándar del grosor de la piel de diabéticos y no diabéticos
info_grosor = datos_preparados["GrosorPiel"].agg(["mean", "std"])
print("Grosor de la piel general:\n", info_grosor)

      Grosor de la piel general:
      mean      27.808011
      std       9.304910
      Name: GrosorPiel, dtype: float64

grosor_diabeticos = df_diabeticos["GrosorPiel"].agg(["mean", "std"])
print("Grosor de la piel de diabéticos:\n", grosor_diabeticos)

      Grosor de la piel de diabéticos:
      mean      30.381526
      std       9.660101
      Name: GrosorPiel, dtype: float64
```

Tiene sentido que los diabéticos tengan de media un mayor grosor de piel, especialmente si la diabetes es de larga evolución o está mal controlada. Esto refleja los cambios estructurales inducidos por la hiperglucemia crónica en el tejido conectivo.

¿Qué porcentaje de pacientes con Glucosa > 140 mg/dL son diabéticos?

```
pacientes_glucosa_alta = datos[datos["Glucosa"] > 140]
porcentaje_diabeticos =
pacientes_glucosa_alta["Resultado"].mean() * 100
```

```
print(f"\nPorcentaje: {porcentaje_diabeticos:.2f}%")  
  
Porcentaje: 68.75%
```

En vez de usar las funciones count y sum, lo hemos sacado con la media porque la columna resultados solo contiene valores 0 y 1. La media es el resultado de dividir la suma de todos los valores de la columna entre el número total de elementos de la columna. Por lo tanto nos sirve para obtener el porcentaje de diabéticos que superan esos valores de glucosa.

Gráficos

De tartas

Gráfico de proporción según [resultado](#) de diabetes (0 = no diabético, 1 = diabético)

```
plt.figure(figsize=(8, 6))  
  
plt.pie(conteo,  
        labels=["No diabéticos", "Diabéticos"],  
        autopct="%.2f%%",  
        colors=["orange", "cornflowerblue"],  
        startangle=90)  
plt.title("Proporción de diabéticos vs no diabéticos")  
  
plt.show()
```

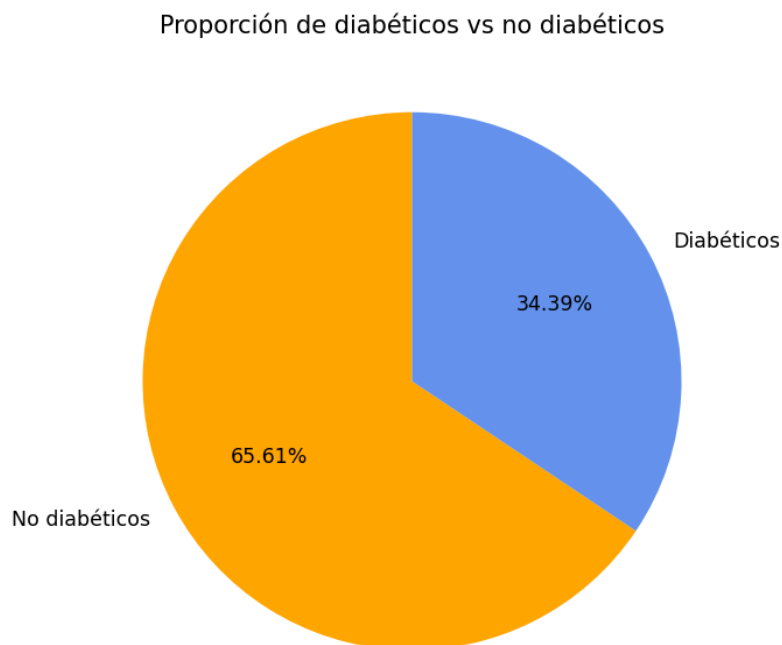


Gráfico de proporción según número de embarazos agrupándolo **por categorías**.

```
# Categorías para embarazos
bins = [0, 1, 3, 5, 10, 20]
labels = ["0", "1-2", "3-4", "5-9", "10+"]

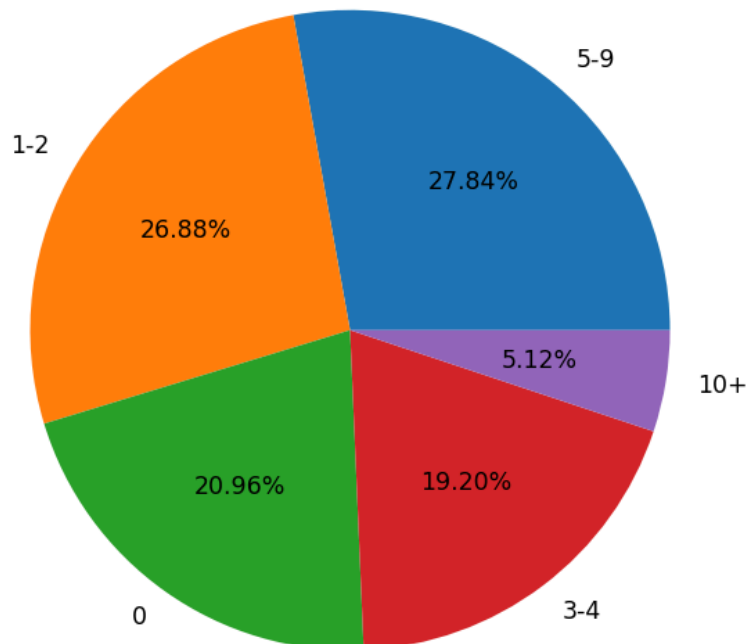
datos_preparados["Embarazos agrupados"] = pd.cut(datos_preparados["Embarazos"],
                                                  bins=bins, labels=labels)

embarazos_counts = datos_preparados["Embarazos
agrupados"].value_counts()

# Gráfico
plt.figure(figsize=(8, 6))
plt.pie(embarazos_counts,
        labels=embarazos_counts.index,
        autopct="%.2f%%")
plt.title("Distribución según número de embarazos (agrupados)")

plt.show()
```

Distribución según número de embarazos (agrupados)



Los gráficos de tartas son una forma muy visual de resaltar qué categorías son más frecuentes, facilitando la comparación entre grupos de un solo vistazo.

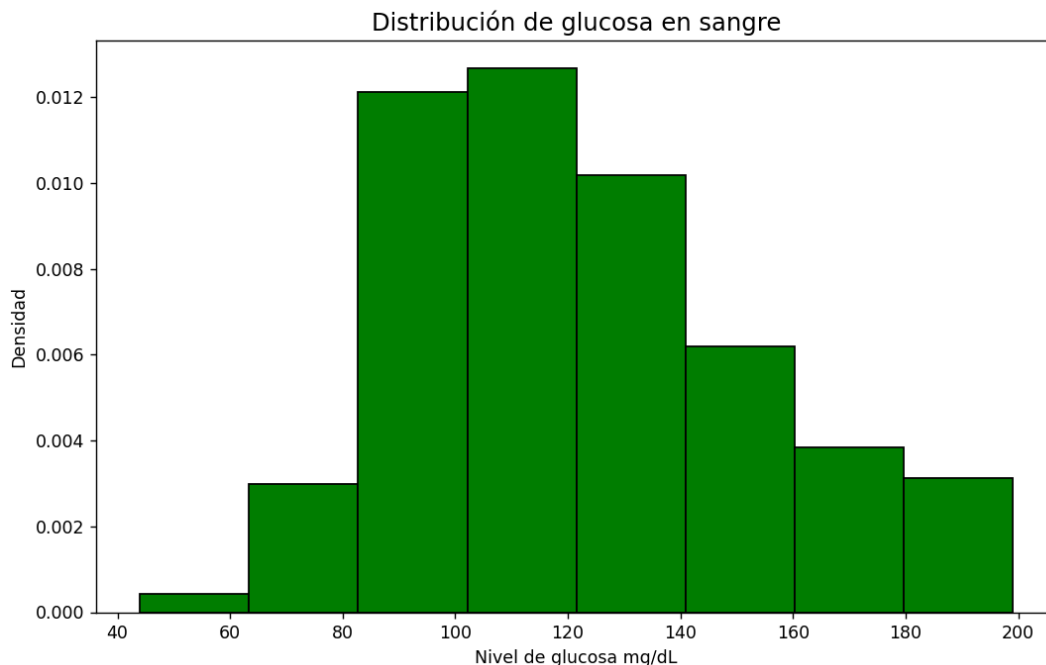
Histograma

Histograma de glucosa en sangre, que muestre la **densidad** de la distribución de probabilidad, con 8 barras.

Cuando density es False, el histograma muestra el conteo absoluto de pacientes en cada rango. Al poner la densidad a true, el histograma muestra la densidad de probabilidad, donde el área total de todas las barras suma 1.

```
plt.figure(figsize=(10, 6))
plt.hist(
    datos_preparados["Glucosa"],
    bins=8,          # 8 barras
    density=True,    # Mostrar densidad de probabilidad
    edgecolor="black", # Bordes de las barras
    color="green",    # Color verde
)

plt.title("Distribución de glucosa en sangre", fontsize=14)
plt.xlabel("Nivel de glucosa mg/dL")
plt.ylabel("Densidad")
plt.show()
```



La mayoría de los valores se concentran entre 90 y 130 mg/dL, lo que sugiere que es el rango más común. Además, la forma de la distribución es asimétrica hacia la

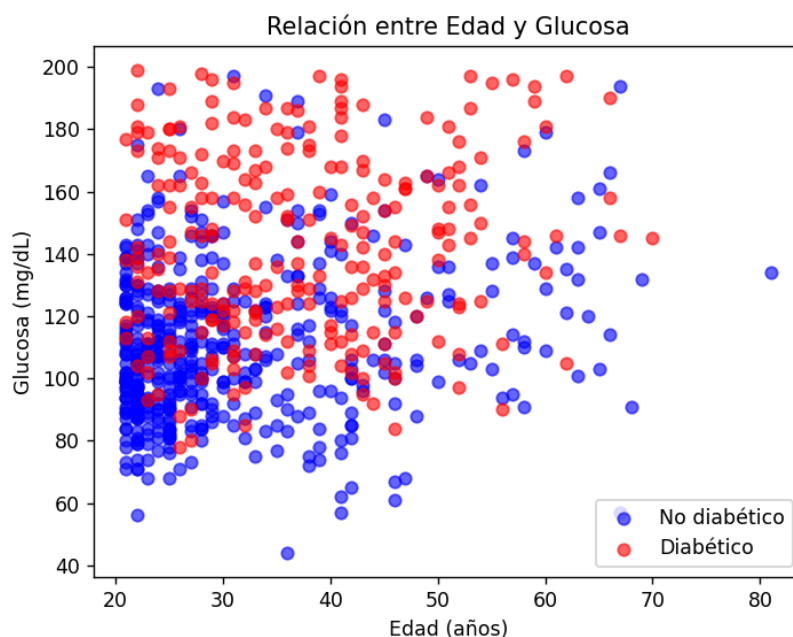
derecha, indicando que hay algunos pacientes con niveles bastante elevados de glucosa.

Nube de puntos

Vamos a representar una nube de puntos con las edades (eje X) y los valores de glucosa (eje Y), separando por diabéticos y no diabéticos.

```
plt.scatter(
    datos_preparados[datos_preparados["Resultado"] == 0]["Edad"],
    datos_preparados[datos_preparados["Resultado"] == 0]["Glucosa"],
    color="blue",
    alpha=0.6,
    label="No diabético"
)
plt.scatter(
    datos_preparados[datos_preparados["Resultado"] == 1]["Edad"],
    datos_preparados[datos_preparados["Resultado"] == 1]["Glucosa"],
    color="red",
    alpha=0.6,
    label="Diabético"
)

plt.title("Relación entre Edad y Glucosa")
plt.xlabel("Edad (años)")
plt.ylabel("Glucosa (mg/dL)")
plt.legend()
plt.show()
```

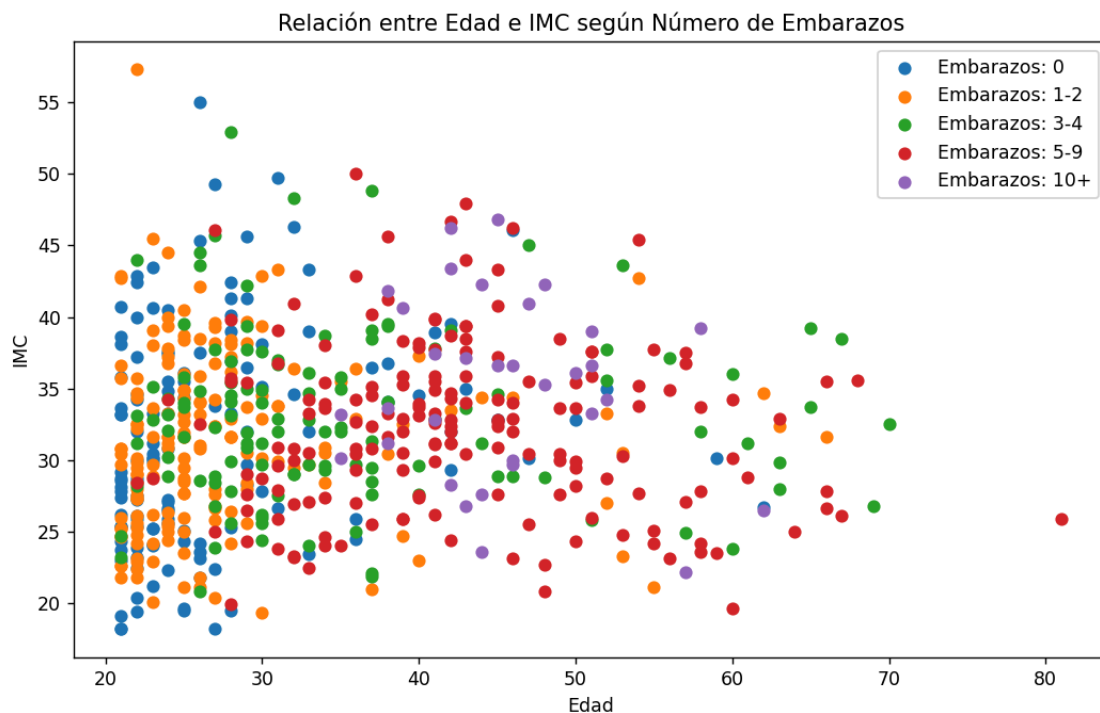


Aunque no hay una tendencia lineal clara, se aprecia que los pacientes diabéticos tienden a tener niveles de glucosa más altos a cualquier edad. También se ve que los no diabéticos se concentran más en valores bajos de glucosa y en edades más jóvenes.

Vamos a representar una nube de puntos con las edades (eje X) y el IMC (eje Y), separando por [Número de Embarazos agrupados](#).

```
plt.figure(figsize=(10, 6))
for grupo in labels:
    subset = datos_preparados[datos_preparados["Embarazos Grupo"] == grupo]
    plt.scatter(
        subset["Edad"],
        subset["IMC"],
        label=f"Embarazos: {grupo}"
    )

plt.title("Relación entre Edad e IMC según Número de Embarazos")
plt.xlabel("Edad")
plt.ylabel("IMC")
plt.legend()
plt.show()
```



Se observa una gran dispersión, pero se podría hablar de la tendencia de que los grupos con más embarazos (rojo, morado) aparecen en edades más avanzadas.

Regresión lineal

En primer lugar vamos a calcular la **correlación** entre variables, que nos sirve para medir la fuerza y dirección de la relación lineal entre ellas. Para ello todas las columnas deben contener datos numéricos. En este caso como todas las columnas son numéricas no hay problema.

La correlación indica si dos variables tienden a cambiar juntas en la misma dirección (correlación positiva), en direcciones opuestas (correlación negativa), o si no existe relación lineal (correlación cero o cercana a cero).

```
# Calcular matriz de correlación
corr_matrix = datos_preparados[columnas_diabetes].corr().round(2)
print(corr_matrix)
```

	Embarazos	Glucosa	PresionArterial	GrosorPiel	\
Embarazos	1.00	0.13	0.21	0.04	
Glucosa	0.13	1.00	0.22	0.18	
PresionArterial	0.21	0.22	1.00	0.16	
GrosorPiel	0.04	0.18	0.16	1.00	
Insulina	-0.05	0.37	-0.03	0.18	
IMC	0.01	0.22	0.29	0.56	
IndiceRiesgoGenetico	-0.03	0.12	-0.01	0.11	
Edad	0.56	0.26	0.32	0.06	
Resultado	0.22	0.49	0.17	0.20	

	Insulina	IMC	IndiceRiesgoGenetico	Edad	Resultado
Embarazos	-0.05	0.01	-0.03	0.56	0.22
Glucosa	0.37	0.22	0.12	0.26	0.49
PresionArterial	-0.03	0.29	-0.01	0.32	0.17
GrosorPiel	0.18	0.56	0.11	0.06	0.20
Insulina	1.00	0.19	0.09	-0.01	0.16
IMC	0.19	1.00	0.09	0.02	0.30
IndiceRiesgoGenetico	0.09	0.09	1.00	-0.02	0.12
Edad	-0.01	0.02	-0.02	1.00	0.25
Resultado	0.16	0.30	0.12	0.25	1.00

La mayor correlación se da entre el grosor de piel y el IMC, empatada con la de edad y embarazos. La menor correlación se da entre el IMC y embarazos y entre edad e insulina.

Para predecir si una persona es diabética o no, se recomienda usar variables que tengan la correlación con Resultado alta y correlaciones bajas entre sí. Las correlaciones mayores en Resultado se dan con la Glucosa (0.49), el IMC(0.3) y la Edad (0.25).

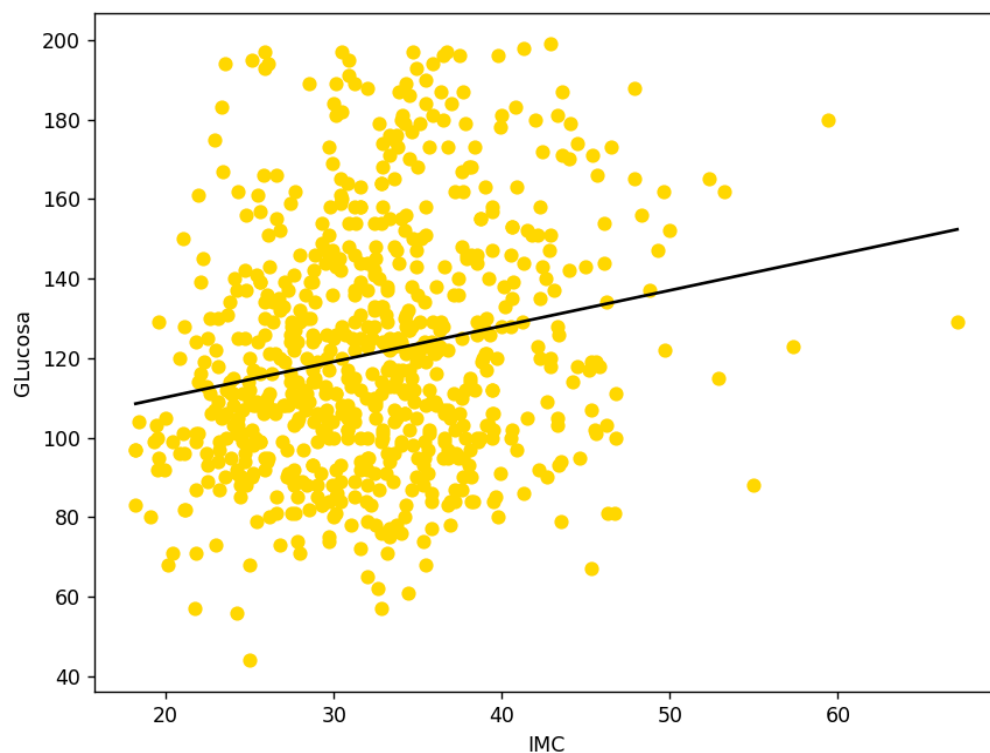
Procedemos comprobar lo que hemos hablado de la correlación generando una nube de puntos con el IMC en el eje X y la glucosa en el eje Y. También hay que calcular la recta de regresión entre ambas variables y la mostrarla en el gráfico.

```
# Regresión lineal
x = datos["IMC"].values.reshape((-1, 1))
y = datos["Glucosa"].values

# Recta de regresión
regression = linear_model.LinearRegression()
modelo = regression.fit(x, y)

# Generamos valores para mostrar la recta de regresión
rango = np.linspace(min(datos_preparados["IMC"]), max(datos_preparados["IMC"]), 100)
predicciones_rango = modelo.predict(rango.reshape(-1, 1))

# Mostramos la recta de regresión
plt.figure(figsize=(8, 6))
plt.scatter(datos_preparados["IMC"], datos_preparados["Glucosa"], color="gold")
plt.plot(rango, predicciones_rango, color="black")
plt.xlabel("IMC")
plt.ylabel("Glucosa")
plt.show()
```



```
# Mostrar métricas
print(f"Pendiente: {regresion.coef_[0]:.2f}")
print(f"Intercepto: {regresion.intercept_:.2f}")
print(f"R²: {regresion.score(x, y):.2f}") # Bondad del ajuste

Pendiente: 0.90
Intercepto: 92.21
R²: 0.05
```

Un R^2 de 0.05 significa que el IMC solo explica el 5% de la variabilidad en los niveles de glucosa en sangre, lo que implica que el 95% restante se debe a otros factores no considerados en el modelo (ej: la genética, la dieta o la actividad física). Este valor de R^2 bajo es esperado porque la correlación entre estas dos variables es baja (0.22) y por ello una relación lineal débil.

A pesar de un R^2 bajo, el modelo sigue siendo útil si el objetivo es identificar tendencias generales (por ejemplo, "A mayor IMC, mayor glucosa en promedio") o confirmar que el IMC tiene cierto impacto, aunque sea pequeño. Sin embargo, no sería adecuado para predecir con precisión los niveles de glucosa individuales, ya que se necesitarían más variables para lograr una mayor exactitud.

Calculamos el error del modelo

Lo vamos a hacer de forma automática con las funciones de `sklearn.metrics`

```
from sklearn.metrics import mean_squared_error,
root_mean_squared_error

# Calcular errores automáticamente
mse = mean_squared_error(y, modelo.predict(x)) # Error Cuadrático Medio
rmse = root_mean_squared_error(y, modelo.predict(x)) # Raíz del mse

print("\nMétricas de error:")
print(f"- MSE: {mse:.2f}")
print(f"- RMSE: {rmse:.2f} (en las mismas unidades que 'Glucosa')")

Métricas de error:
- MSE: 971.02
- RMSE: 31.16 (en las mismas unidades que 'Glucosa')
```

El MSE mide el error promedio al cuadrado entre tus predicciones y los valores reales. Con este cálculo descubrimos que, en promedio, las predicciones se desvían 31,16 mg/dL de los valores reales.

Generar divisiones de los datos originales con train_test_split

Para terminar vamos a entrenar un modelo de regresión lineal múltiple. Vamos a intentar predecir la presión arterial en función de edad, embarazos, IMC y glucosa.

Con regresiones lineales hay que intentar predecir valores continuos. No es recomendable intentar predecir el valor de la columna Resultado porque sólo admite valores binarios: 0 ó 1. Para predicciones de este tipo sería más recomendable una regresión logística, pero esa regresión no va a ser contenido evaluable así que nos quedamos con el enunciado del párrafo previo.

Para hacerlo un poco más interesante, vamos a dividir los datos originales: El 90% de los valores van a 'entrenamiento' y el 10% restante van en 'test'. Necesitamos hacer uso de train_test_split:

```
from sklearn.model_selection import train_test_split
```

Dividir los datos es como preparar un examen para tu modelo de predicción. Con el 90% le enseñas cómo funcionan los datos (como estudiar con un libro de texto), y con el 10% lo pruebas para ver si realmente aprendió (como hacer un examen). Así sabes si tu modelo puede predecir bien casos nuevos que nunca ha visto antes.

```
# Regresión lineal múltiple
# Queremos predecir la presión arterial a partir de otras variables
x = datos_preparados[["Edad", "Embarazos", "IMC", "Glucosa"]]
y = datos_preparados["PresionArterial"]

# Dividir datos en entrenamiento (90%) y test (10%)
x_entrenamiento, x_test, y_entrenamiento, y_test = train_test_split(
    x,
    y,
    test_size=0.1,
    random_state=42
)

print(f"Filas entrenamiento: {x_entrenamiento.shape[0]}")
print(f"Filas test: {x_test.shape[0]}")

        Filas entrenamiento: 651
        Filas test: 73

# Procedemos con el modelo
modelo_lineal = linear_model.LinearRegression()
modelo_lineal.fit(x_entrenamiento, y_entrenamiento)
```

```

# Predicción
y_pred = modelo_lineal.predict(x_test)

# Evaluación
mse = mean_squared_error(y_test, y_pred)
rmse = root_mean_squared_error(y_test, y_pred)

print("\nEvaluación del modelo de regresión lineal múltiple:")
print(f"- MSE: {mse:.2f}")
print(f"- RMSE: {rmse:.2f}")
print(f"- R²: {modelo_lineal.score(x_test, y_test):.2f}")

```

```

Evaluación del modelo de regresión lineal múltiple:
- MSE: 106.56
- RMSE: 10.32
- R²: 0.23

```

La función `train_test_split` recibe por un lado los datos independientes (x) y la variable a predecir (y). Luego está el parámetro `test_size=0.1` que indica el porcentaje destinado a test (el 10% en este caso). Por último, `random_state=42` asegura que la partición sea reproducible, es decir, se asemeja a ponerle una contraseña fija a la división de datos. Si otro usa el mismo número (42), obtendrá exactamente los mismos grupos de entrenamiento y prueba que la nuestra, lo que facilita comparar resultados.

El grupo de entrenamiento (`x_entrenamiento`, `y_entrenamiento`) se usa para que el modelo aprenda la relación entre las variables (edad, IMC...) y lo que queremos predecir (presión arterial). Este aprendizaje es el ajuste del modelo de regresión. Una vez aprendido, probamos el modelo con datos que no ha visto (`x_test`) y comparamos las predicciones con los valores reales (`y_test`). Esto sirve para ver si el modelo generaliza bien y no solo "se ha memorizado" los datos de entrenamiento.

El modelo de regresión lineal múltiple ha obtenido un R^2 de 0.23, lo que indica que aproximadamente el 23% de la variabilidad en la presión arterial puede explicarse por las variables utilizadas. Aunque es una mejora respecto al modelo simple de glucosa con IMC (que solo explicaba el 5%), el ajuste sigue siendo bajo. El RMSE de 10.32 refleja el error medio en las predicciones. En resumen, el modelo capta cierta tendencia, pero no es suficientemente preciso para predicciones individuales.

¡¡Y con esto damos por finalizado el contenido evaluable de la asignatura!!
Este documento sirve de guía para la tarea del tema 3.
Mucho ánimo que ya queda poco.